04. 도구(Tools) 시스템

이 장에서는 Hermes Agent의 핵심인 도구(Tools) 시스템을 살펴봅니다. 40개 이상의 내장 도구와 6가지 터미널 백엔드를 이해하고, 실습을 통해 도구 활용 방법을 익힙니다.

이 장에서 다루는 내용

- 내장 도구 살펴보기
- 터미널 백엔드
- 실습: 도구를 활용한 작업

04-1. 내장 도구 살펴보기

> ⚠️ ***부분 검증**: 이 페이지는 v0.16.0 소스 코드(`tools/registry.py` 가 `tools/*.py` 의 `registry.register` 호출을 수집해 등록)를 기준으로 점검했습니다. 도구 구성은 버전마다 바뀌므로(예: RL 학습 도구는 제거됨), 정확한 현재 활성 목록은 `hermes tools` 로 확인하세요. 실행 테스트는 거치지 않았습니다.*

- 도구 세트 개요
- 도구 활성화와 관리
- 전체 도구 목록
  - 웹 (3개)
  - 터미널 (2개)
  - 파일 (4개)
  - 브라우저 (12개)
  - 미디어 (5개)
  - 에이전트 오케스트레이션 (5개)
  - 메모리 및 검색 (2개)
  - 스킬 관리 (3개)
  - 자동화 (2개)
  - Kanban 협업 보드 (9개)
  - Computer Use (1개)
  - Discord (2개)
  - Feishu (5개)
  - Yuanbao (5개)
  - Home Assistant (4개)
- 주요 도구 작동 방식
  - todo — 에이전트의 스크래치패드
  - process — 백그라운드 프로세스 관리
  - 파일 도구의 변경 감지와 진단
  - 브라우저 백엔드 선택
  - 이미지·동영상 생성 백엔드
  - vision_analyze — 비전 모델에 원본 픽셀 직접 전달
- API 키와 Nous Tool Gateway
- 버전별 주요 변경 사항

도구 세트 개요

Hermes Agent의 도구는 **도구 세트(toolset)** 단위로 묶여 있으며, 도구 세트별로 활성화하거나 비활성화할 수 있습니다. 모든 도구는 `tools/registry.py`에 등록되고, 활성화 상태는 CLI 옵션과 슬래시 명령어로 관리합니다. 현재 활성화된 도구 목록은 `hermes tools`로 확인할 수 있습니다.

도구 활성화와 관리

```
hermes tools                          # 활성 도구·세트 보기
/tools                                 # 대화 중 도구 활성/비활성 토글
hermes chat --toolsets "terminal,file,web"   # 특정 세트만 활성화
```

다음 도구 세트는 기본적으로 **비활성**입니다(`hermes_cli/tools_config.py`의 `_DEFAULT_OFF_TOOLSETS`): `moa`, `homeassistant`, `spotify`, `discord`, `discord_admin`, `video`, `video_gen`, `x_search`. `--toolsets`로 명시하거나 `/tools`에서 켜야 사용할 수 있습니다(단, `homeassistant`은 `HASS_TOKEN`이 설정돼 있으면 자동으로 켜집니다).

전체 도구 목록

웹 (3개)

도구	기능
`web_search`	웹 검색. `limit` 파라미터로 결과 개수 조정
`web_extract`	웹 페이지 콘텐츠 추출·파싱
`x_search`	X(Twitter) 검색 — 타임라인·스레드·특정 게시물 조회. xAI OAuth 또는 API 키로 인증

무료 검색 백엔드

유료 API(Firecrawl·Tavily·Exa·Parallel) 외에 SearXNG·DuckDuckGo(`ddgs`)·Brave Search 무료 티어를 백엔드로 쓸 수 있습니다. SearXNG는 셀프호스트 메타검색 엔진으로, `SEARXNG_URL` 환경 변수만 설정하면 됩니다. xAI(Grok)의 웹 검색도 백엔드로 고를 수 있으며, 이때는 xAI 자격증명이 필요합니다(`search_backend: xai`로 명시 지정).

터미널 (2개)

도구	기능
`terminal`	셸 명령 실행
`process`	백그라운드 프로세스 관리 (목록·로그·종료·입력·완료 알림·패턴 감지)

파일 (4개)

도구	기능
`read_file`	파일 읽기
`write_file`	파일 쓰기
`patch`	파일 편집(패치 적용)
`search_files`	파일 검색

브라우저 (12개)

Playwright 기반 헤드리스 브라우저 제어 도구입니다.

도구	기능
`browser_navigate`	URL로 이동
`browser_snapshot`	페이지 스냅숏 캡처
`browser_click`	요소 클릭
`browser_type`	텍스트 입력
`browser_scroll`	페이지 스크롤
`browser_back`	뒤로 가기
`browser_press`	키 입력
`browser_get_images`	페이지 내 이미지 추출
`browser_vision`	페이지의 시각적 분석
`browser_console`	브라우저 콘솔 로그 조회
`browser_cdp`	Chrome DevTools Protocol 패스스루 — 메인 도구가 커버하지 않는 저수준 제어
`browser_dialog`	네이티브 JS 다이얼로그(alert/confirm/prompt) 응답

미디어 (5개)

도구	기능
`vision_analyze`	이미지 분석(비전 모델 사용)
`video_analyze`	동영상 프레임 추출·분석
`image_generate`	이미지 생성
`video_generate`	동영상 생성 — 플러그인 백엔드로 모델 교체
`text_to_speech`	텍스트를 음성으로 변환

에이전트 오케스트레이션 (5개)

도구	기능
`todo`	작업 계획·추적(인메모리 태스크 리스트)
`clarify`	사용자에게 확인 질문
`execute_code`	Python 코드 실행
`delegate_task`	서브에이전트에 작업 위임
`mixture_of_agents`	여러 모델의 응답을 종합

메모리 및 검색 (2개)

도구	기능
`memory`	영속 메모리 읽기/쓰기
`session_search`	과거 세션 전문 검색

스킬 관리 (3개)

도구	기능
`skills_list`	설치된 스킬 목록 조회
`skill_view`	스킬 상세 내용 확인
`skill_manage`	스킬 생성·편집·삭제

자동화 (2개)

도구	기능
`cronjob`	예약 작업 관리
`send_message`	메시징 플랫폼으로 메시지 전송

Kanban 협업 보드 (9개)

같은 머신에서 여러 Hermes 프로필이 협업할 때 쓰는 영속 칸반입니다. 슬래시 명령 `/kanban`과도 짝을 이룹니다.

도구	기능
`kanban_create`	새 카드 생성
`kanban_list`	보드 상태별·소유자별 카드 조회(오케스트레이터 전용)
`kanban_show`	단일 카드 상세 조회
`kanban_complete`	카드 완료 처리
`kanban_block`	블로커 등록·블록 사유 기록
`kanban_unblock`	블로커 해제(오케스트레이터 전용)
`kanban_heartbeat`	워커가 살아 있음을 보드에 통지
`kanban_comment`	카드에 코멘트 추가
`kanban_link`	카드 간 의존 관계 등록

자세한 다중 에이전트 협업 구조는 「Kanban Multi-Agent 협업」 절에서 다룹니다.

Computer Use (1개)

도구	기능
`computer_use`	macOS 데스크톱 제어(cua-driver 기반). 스크린샷·클릭·타이핑 등을 백그라운드에서 수행 — 사용자의 커서·키보드 포커스를 빼앗지 않습니다. 비전 가능 모델(GPT-5·Claude·Gemini·Grok-vision 등)에서 작동하며 `hermes update` 시 자체 갱신

Discord (2개)

도구	기능
`discord`	메시지 전송·채널 조회·리액션 등 일반 봇 기능
`discord_admin`	서버 관리·역할 조작 등 관리자 전용 기능

Feishu (5개)

도구	기능
`feishu_doc_read`	Feishu(Lark) 문서 읽기
`feishu_drive_list_comments`	드라이브 파일 댓글 목록
`feishu_drive_list_comment_replies`	댓글 답글 목록
`feishu_drive_reply_comment`	댓글에 답글 작성
`feishu_drive_add_comment`	드라이브 파일에 댓글 작성

Yuanbao (5개)

도구	기능
`yb_query_group_info`	Yuanbao 그룹 정보 조회
`yb_query_group_members`	그룹 멤버 목록
`yb_send_dm`	다이렉트 메시지 전송
`yb_search_sticker`	스티커 검색
`yb_send_sticker`	스티커 전송

Home Assistant (4개)

도구	기능
`ha_list_entities`	엔티티 목록 조회
`ha_get_state`	엔티티 상태 조회
`ha_list_services`	사용 가능한 서비스 목록
`ha_call_service`	서비스 호출(조명 켜기·온도 조절 등)

주요 도구 작동 방식

도구별로 알아 두면 좋은 작동 방식을 한곳에 모았습니다.

`todo` — 에이전트의 스크래치패드

코딩 에이전트가 긴 작업을 수행할 때 가장 큰 위험은 **"지금 어디까지 했고, 다음에 뭘 해야 하는지"를 잊는 것**입니다. LLM의 컨텍스트 윈도우는 한계가 있어 대화가 길어지면 오래된 메시지가 압축되거나 잘려 나갑니다.

`todo` 도구는 이 문제를 해결하는 **인메모리 태스크 리스트**입니다. 에이전트는 3단계 이상의 복잡한 작업을 받으면 스스로 호출해 계획을 세우고, 각 단계를 완료할 때마다 상태를 갱신합니다.

```
- [>] 1. API 엔드포인트 구현 (in_progress)
- [ ] 2. 단위 테스트 작성 (pending)
- [ ] 3. README 업데이트 (pending)
```

각 항목은 네 가지 상태를 가집니다.

상태	의미
`pending`	대기 중
`in_progress`	진행 중 (한 번에 하나만)
`completed`	완료
`cancelled`	취소

핵심은 **컨텍스트 압축과의 연동**입니다. 대화가 길어져 압축이 일어나면 에이전트는 완료·취소 항목을 버리고 **활성 항목(pending, in_progress)만** 압축된 컨텍스트에 자동으로 주입합니다. 덕분에 컨텍스트를 잃어도 작업 흐름을 이어갈 수 있습니다.

Claude Code의 `TodoWrite` / `TodoRead` 도 같은 역할입니다. 코딩 에이전트에게 스크래치패드는 선택이 아니라 필수입니다.

`process` — 백그라운드 프로세스 관리

장기 실행 작업을 백그라운드에서 실행할 때, 매번 폴링하지 않아도 됩니다.

- **시작**: `background=true` 플래그로 실행
- **관리**: 목록 조회·로그 확인·대기·종료·입력 쓰기
- `notify_on_complete=True` : 프로세스가 끝나면 자동으로 에이전트에게 알림
- `watch_patterns=["error", "listening on port"]` : 출력에 지정 패턴이 등장하면 즉시 알림. 빌드 로그에서 에러를 잡거나, 서버가 시작돼 특정 메시지를 출력할 때까지 기다리는 용도로 적합

연속 3회 false strike — 패턴이 매칭됐지만 이후 의미 있는 출력이 이어지지 않는 상태가 누적되면, watch가 자동으로 꺼지고 해당 세션은 이후 `notify_on_complete` 알림만 받습니다.

파일 도구의 변경 감지와 진단

파일 도구에는 **변경 감지(stale file detection)** 안전장치가 들어 있습니다. `read_file` 로 파일을 읽으면 Hermes가 수정 시각(mtime)을 기록합니다. 이후 `write_file` 이나 `patch` 를 호출할 때 외부에서 파일이 바뀐 상태라면 경고 메시지를 반환합니다. 쓰기를 막지는 않으며, 에이전트가 파일을 다시 읽도록 알려 주는 약한 경고 역할입니다.

또한 `write_file` · `patch` 를 실행한 뒤에는 **LSP 시맨틱 진단**을 자동으로 실행합니다. Python·JSON·YAML·TOML의 구문 검사뿐 아니라, 실제 언어 서버를 실행해 타입 오류, 정의되지 않은 심볼, 누락된 import 같은 의미 수준의 문제까지 다음 턴 직전에 잡아냅니다. **이번 쓰기로 새로 발생한 오류만** 보고하므로 노이즈가 적습니다.

브라우저 백엔드 선택

기본 백엔드는 agent-browser CLI입니다. `CAMOFOX_URL` 환경 변수를 설정하면 Camoufox(Firefox 포크 기반 안티디텍션 브라우저)로 로컬에서 제어합니다. Browserbase 같은 클라우드 서비스 없이 핑거프린트 우회가 필요한 작업을 처리할 수 있는 무료 대안입니다.

브라우저 도구는 첫 호출 시 사용 가능한 Chromium 계열 브라우저(Chrome·Brave 등)를 자동으로 실행해 CDP로 연결합니다. 사용자가 별도로 브라우저를 띄워 두지 않아도 작동합니다(Hermes 커밋 `697d38a3f`).

이미지·동영상 생성 백엔드

`image_generate` 는 플러그인 레지스트리 구조로 전환되어 FAL 백엔드 하나에서 FLUX 2 Pro·FLUX 2 Klein·Nano Banana Pro·Recraft V4 Pro·Ideogram V3 등 여러 모델을 선택할 수 있습니다. GPT Image 1.5/2, Qwen Image 같은 비-FAL 모

델도 별도 프로바이더로 등록됩니다. `video_generate` 도 같은 플러그인 구조라 백엔드를 갈아 끼울 수 있습니다.

vision_analyze — 비전 모델에 원본 픽셀 직접 전달

비전 가능 모델(GPT-5·Claude·Gemini·Grok-vision)이 활성화돼 있으면 텍스트 설명 변환 없이 **원본 픽셀을 직접 전달**합니다. 텍스트 요약을 거치지 않으므로 모델의 실제 시각 추론을 그대로 활용할 수 있습니다.

API 키와 Nous Tool Gateway

Hermes Agent의 일부 도구(웹 검색·이미지 생성·TTS·브라우저 자동화)는 외부 SaaS API 키가 필요합니다. **유료 Nous Portal 구독자**라면 이런 도구를 별도 키 없이 Nous 구독 한 장으로 사용할 수 있습니다.

도구	백엔드	직접 키 대안
웹 검색·추출	Firecrawl	`FIRECRAWL_API_KEY` 등
이미지 생성	FAL (FLUX 2 Pro·Klein, Nano Banana Pro, Recraft V4 Pro, Ideogram V3, Z-Image Turbo 등)	`FAL_KEY`
TTS	OpenAI TTS	`VOICE_TOOLS_OPENAI_KEY`
브라우저 자동화	Browser Use	`BROWSER_USE_API_KEY`

활성화는 `hermes tools` 메뉴에서 도구별로 **Nous Subscription** 프로바이더를 고르거나, `config.yaml` 에서 `use_gateway: true` 를 직접 설정합니다.

```
web:
  backend: firecrawl
  use_gateway: true
image_gen:
  use_gateway: true
tts:
  provider: openai
  use_gateway: true
browser:
  cloud_provider: browser-use
  use_gateway: true
```

`use_gateway: true` 이면 직접 키가 `.env` 에 있어도 게이트웨이 경유가 우선합니다. 이 옵션은 무료 티어에서는 작동하지 않습니다.

버전별 주요 변경 사항

- **v0.14.0** — `x_search` · `video_generate` 신규 도구. `vision_analyze` 가 비전 가능 모델에 원본 픽셀 직접 전달. `computer_use` cua-driver가 비-Anthropic 모델에서도 작동, `hermes update` 시 자체 갱신. LSP 시맨틱 진단이 구문 검사에서 의미 분석(타입 오류·정의되지 않은 심볼·누락된 import)으로 확장.
- **v0.13.0** — LSP 시맨틱 진단(`83b93898c`)과 쓰기 후 델타 린트(`5168226d6`) 도입. Honcho 관련 4개 도구 (`honcho_search` , `honcho_context` , `honcho_profile` , `honcho_conclude`) 제거(`55107a196`).
- **v0.12.0 이전** — 도구 백엔드 일부에 무료 검색 옵션(`5c906d702`), `browser_cdp` · `browser_dialog` (`ce410521b`), `video_analyze` (`c9a3f36f5`), `computer_use` 백그라운드 실행(`850413f12`), `discord_admin` (`ef73367fc`) 추가.

04-2. 터미널 백엔드

> ⚠️ ***미검증**: 이 절은 v0.13.0 공식 문서·소스 코드를 바탕으로 작성되었습니다. 실제 테스트 후 업데이트될 예정입니다.*

- 백엔드 종류
- local (기본)
- docker
- ssh
- modal / daytona / vercel_sandbox (서버리스)
- 브라우저 백엔드와 자동 로컬 폴백
- 어떤 백엔드를 선택할 것인가

Hermes Agent는 7가지 터미널 백엔드를 지원하여, 명령 실행 환경을 유연하게 선택할 수 있습니다.

백엔드 종류

백엔드	설명	보안 수준
local	호스트 시스템에서 직접 실행	낮음
docker	Docker 컨테이너 내에서 실행	높음
ssh	원격 서버에 SSH 접속하여 실행	높음
singularity	Singularity 컨테이너에서 실행	높음
modal	Modal 서버리스 환경에서 실행	높음
daytona	Daytona 서버리스 워크스페이스	높음
vercel_sandbox	Vercel Sandbox 클라우드 샌드박스에서 실행	높음

local (기본)

가장 간단한 백엔드로, 에이전트가 호스트 시스템에서 직접 명령을 실행합니다.

```
terminal:
  backend: local
  cwd: "."
  timeout: 180
  persistent_shell: true
```

장점: 빠른 실행, 설정 불필요 **단점**: 에이전트가 시스템 전체에 접근 가능하므로 보안 위험

docker

Docker 컨테이너 내에서 격리된 환경을 제공합니다.

```
terminal:
  backend: docker
  docker_image: "nikolaik/python-nodejs:python3.11-nodejs20"
  docker_mount_cwd_to_workspace: false
  docker_volumes:
    - "/host/path:/container/path"
```

컨테이너 보안 설정:

- 읽기 전용 루트 파일시스템
- 권한 상승(privilege escalation) 차단
- PID 제한 (256)
- 전체 네임스페이스 격리
- CPU, 메모리, 디스크 제한 설정 가능
- **호스트 사용자 ID로 컨테이너 실행** — 기본적으로 컨테이너의 사용자 UID/GID가 호스트와 같아져 바인드 마운트한 작업 디렉터리에 root 소유 파일이 만들어지지 않습니다. 컨테이너에서 만든 파일을 호스트 셸에서 그대로 편집할 수 있습니다.

ssh

에이전트가 원격 서버에서 명령을 실행합니다. 에이전트 자체의 코드를 수정할 수 없으므로 보안 면에서 권장되는 백엔드입니다.

```
terminal:
  backend: ssh
  ssh_host: "user@remote-server"
  ssh_key: "~/.ssh/id_rsa"
```

modal / daytona / vercel_sandbox (서버리스)

서버리스 백엔드는 유휴 시 환경이 잠들어 비용이 거의 발생하지 않고, 필요 시 깨어나 작업을 수행합니다.

- **Modal**: GPU 인스턴스 포함 가능, 초 단위 과금
- **Daytona**: 개발 워크스페이스 관리에 특화
- **Vercel Sandbox** (`5a1d4f680`): Vercel이 제공하는 격리 샌드박스. 기본 작업 디렉터리는 `/vercel/sandbox` 이며, 런타임으로 `node24` , `node22` , `python3.13` 을 지원합니다. Vercel 계정이 있고 빠른 시작·짧은 라이프사이클이 어울리는 일감(코드 실행, 단발성 작업 등)에 적합합니다

브라우저 백엔드와 자동 로컬 폴백

터미널은 아니지만, 브라우저 도구도 같은 결의 백엔드 선택이 있습니다. 클라우드 브라우저(Browserbase·Browser Use 등)가 설정돼 있어도 LAN/localhost URL이 들어오면 **로컬 Chromium을 자동 스폰**해 거기서 처리합니다. 사내망의 페이지를 클라우드로 보내지 않으려는 의도이며, `browser.auto_local_for_private_urls` 옵션으로 끌 수 있습니다.

v0.13.0에서는 로컬 브라우저 엔진으로 **Lightpanda**를 선택할 수 있습니다(`395dbcc87`). Lightpanda는 네비게이션이 1.3~5.8배 빠르지만 스크린샷을 지원하지 않으므로, 시각 분석이 필요 없는 웹 스크래핑에 적합합니다. Lightpanda가 처리할 수 없는 요청은 Chrome으로 자동 폴백됩니다.

```
browser:
  engine: lightpanda   # auto(기본, Chrome) | lightpanda | chrome
```

어떤 백엔드를 선택할 것인가

- **개인 학습/실험**: `local` 이면 충분
- **신뢰할 수 없는 코드 실행**: `docker`
- **보안이 중요한 운영 환경**: `ssh` 또는 `docker`
- **비용 최적화**: `modal` , `daytona` , `vercel_sandbox`
- **HPC 환경**: `singularity`

이 책의 실습에서는 `local` 백엔드를 사용합니다. 간단한 실험과 학습에는 이것으로 충분하며, 이후 보안 장(11장)에서 컨테이너 격리를 다룹니다.

04-3. 실습: 도구를 활용한 작업

- 실습 1: 터미널 도구로 프로젝트 분석
- 실습 2: 파일 읽기와 패치
- 실습 3: 웹 검색과 정보 추출
- 실습 4: 코드 실행
- 실습 5: 서브에이전트 위임
- 실제 테스트 결과
  - 터미널 도구 (실습 1): 성공
  - 파일 읽기와 패치 (실습 2): 성공
  - 웹 검색 (실습 3): 실패
  - 코드 실행 (실습 4): execute_code 실패, 터미널 우회 성공
  - 서브에이전트 (실습 5)
- 도구별 로컬 LLM 호환성 요약
- 관찰 포인트

이 절에서는 Hermes Agent의 다양한 도구를 실제로 사용해 봅니다.

실습 전 준비

`/verbose`를 실행하여 도구 호출 과정이 표시되도록 설정하세요. 에이전트가 어떤 도구를 어떤 순서로 호출하는지 관찰하는 것이 학습에 큰 도움이 됩니다.

실습 1: 터미널 도구로 프로젝트 분석

```
> 현재 디렉터리의 파일 구조를 트리 형태로 보여주고,
> 총 파일 수와 디렉터리 수를 알려줘
```

에이전트가 `terminal` 도구를 사용하여 `find`나 `ls` 명령을 실행하고 결과를 분석합니다.

실습 2: 파일 읽기와 패치

```
> ~/.hermes/config.yaml 파일을 읽어서 현재 설정 내용을 설명해줘
```

그 후:

```
> compression.threshold를 0.60으로 변경해줘
```

에이전트가 `read_file` 도구로 파일을 읽고, `patch` 도구로 수정합니다.

실습 3: 웹 검색과 정보 추출

```
> "Hermes Agent NousResearch" 관련 최신 기사를 검색하고,
> 핵심 내용을 3줄로 요약해줘
```

`web_search` → `web_extract` → 요약의 도구 체인을 관찰할 수 있습니다.

실습 4: 코드 실행

```
> Python으로 피보나치 수열의 처음 20개 항을 계산하고 출력해줘
```

에이전트가 `execute_code` 도구를 사용하여 Python 스크립트를 작성하고 실행합니다.

실습 5: 서브에이전트 위임

```
> /background "이 시스템에 설치된 Python 패키지 목록을 분석하고, 보안 취약점이 있는 패키지가 있는지 확인해줘"
```

메인 대화를 이어가면서 서브에이전트가 백그라운드에서 분석 작업을 수행합니다.

실제 테스트 결과

위 실습들을 macOS Intel Mac 환경에서 Qwen 2.5 14B(Ollama)로 실행한 결과입니다. 현재 권장 모델은 Qwen3 14B입니다(부록「로컬 LLM 가이드」참조).

터미널 도구 (실습 1): 성공

에이전트가 `find`, `ls -la` 등의 명령을 올바르게 조합하여 디렉터리 구조를 분석했습니다. 터미널 도구는 로컬 LLM에서 가장 안정적으로 작동하는 도구입니다.

파일 읽기와 패치 (실습 2): 성공

`read_file`로 설정 파일을 읽고 `patch` 도구로 수정하는 과정이 정상 작동했습니다. 14B 모델은 파일 내용을 정확히 이해하고 필요한 부분만 수정할 수 있었습니다.

웹 검색 (실습 3): 실패

`web_search` 도구 호출 후 에이전트가 `<think>` 블록만 반복 출력하고, 실제 응답을 생성하지 못했습니다. 3회 재시도 모두 동일한 실패였습니다. 로컬 LLM은 웹 검색 결과처럼 외부에서 들어오는 대량의 텍스트를 처리하는 데 한계가 있습니다.

코드 실행 (실습 4): execute_code 실패, 터미널 우회 성공

`execute_code` 도구를 사용한 직접 코드 실행은 10회 연속 실패했습니다. 원인은 `\n` 이스케이프 문자 처리 버그로, 모델이 생성하는 코드의 줄바꿈이 깨져서 실행할 수 없는 코드가 만들어졌습니다.

우회 방법으로, 먼저 `write_file`로 `.py` 파일을 생성한 뒤, 터미널에서 `python3 파일명.py`를 실행하면 안정적으로 작동했습니다.

```
> /tmp/fibonacci.py 파일에 피보나치 수열 코드를 작성하고,
> 터미널에서 python3로 실행해줘
```

로컬 LLM에서의 코드 실행 권장 방법

로컬 LLM에서 `execute_code` 도구는 이스케이프 문자 처리 문제로 신뢰성이 낮습니다. 코드를 실행해야 할 때는 다음 순서를 권장합니다.

1. `write_file` 도구로 `.py` 파일 생성
2. `terminal` 도구로 `python3 파일명.py` 실행

이 방식이 훨씬 안정적이며, 코드 파일이 남아 디버깅에도 유리합니다.

서브에이전트 (실습 5)

서브에이전트 기능은 메인 에이전트의 도구 호출 능력에 의존합니다. 14B 모델에서는 기본적인 백그라운드 작업이 작동하지만, 복잡한 분석 작업에서는 모델 능력의 한계로 결과 품질이 떨어질 수 있습니다.

도구별 로컬 LLM 호환성 요약

도구	Qwen 2.5 14B 결과	비고
terminal	안정적	가장 신뢰할 수 있는 도구
write_file / read_file	안정적	실제 경로 사용 (14B 이상)
patch	안정적	파일 수정 정상 작동
execute_code	불안정	`\n` 처리 버그로 실패 빈번
web_search	실패	응답 생성 불가

관찰 포인트

- 에이전트가 도구를 선택하는 기준은 무엇인가?
- 도구 실행이 실패했을 때 에이전트는 어떻게 대처하는가? `execute_code` 실패 시 터미널 우회를 스스로 시도하는지 관찰하세요.
- 여러 도구를 조합할 때 순서와 논리는 어떻게 결정되는가?
- 로컬 8B 모델과 14B 모델에서 도구 호출 정확도가 어떻게 다른가?

04-4. Codex App-Server 런타임

> ⚠️ ***부분 검증**: 이 절은 v0.16.0 소스 코드(`hermes_cli/codex_runtime_switch.py` 등)와 공식 문서를 대조해 작성했으며, 실행 테스트는 거치지 않았습니다. Codex App-Server 런타임은 v0.14.0에서 도입돼 이후 보완됐고, 업스트림에서는 여전히 옵트인 베타로 표시돼 있습니다.*

- Codex App-Server 런타임이란
- 사전 준비
- 활성화와 비활성화
- 사용 가능한 도구의 변화
- 기본 런타임과 비교
- 샌드박스 권한
- 신뢰성 — 세션 재사용·자동 회수·OAuth 갱신
- 언제 쓰고, 언제 끄나

4-2절에서 살펴본 터미널 백엔드(Docker, SSH 등)가 "명령을 어디서 실행할지"를 바꾸는 것이라면, 이 절에서 다루는 Codex 런타임은 **도구 루프 자체를 교체**하는 더 큰 변화입니다.

Codex App-Server 런타임이란

Hermes Agent는 기본적으로 자체 도구 루프(ReAct 루프)로 모델 응답을 처리합니다. Codex App-Server 런타임을 활성화하면, OpenAI/Codex 모델의 턴을 OpenAI Codex CLI의 `app-server`에 위임합니다. 터미널 명령, 파일 편집, 샌드박싱은 Codex가 처리하고, Hermes는 세션 관리·메모리·스킬 리뷰·게이트웨이·크론 같은 외곽 역할을 합니다.

핵심 가치는 두 가지입니다.

- **ChatGPT 구독 인증**: API 키 없이 ChatGPT 구독으로 OpenAI 모델을 사용할 수 있습니다.
- **Codex 네이티브 플러그인**: Linear, GitHub, Gmail, Google Calendar, Canva 등 Codex CLI에 설치된 플러그인을 Hermes 세션에서 그대로 씁니다.

이 런타임은 **옵트인 전용**입니다. 명시적으로 켜지 않는 한 Hermes의 기본 작동은 바뀌지 않습니다.

사전 준비

1. Codex CLI 설치

```
npm i -g @openai/codex
codex --version   # 0.130.0 이상
```

2. Codex 로그인

```
codex login       # ~/.codex/auth.json에 OAuth 토큰 저장
```

> ⚠️ ***Hermes 인증과 별개입니다.*** *`hermes auth add openai-codex` 는 Hermes 자체의 인증이고, `codex login` 은 Codex CLI의 인증입니다. 두 세션은 의도적으로 분리되어 있으므로 각각 로그인해야 합니다.*

3. (선택) Codex 플러그인 설치

```
codex plugin marketplace add openai-curated
# Codex TUI에서 Linear / GitHub / Gmail 등 설치
```

활성화와 비활성화

세션 도중에 슬래시 명령으로 전환합니다.

```
> /codex-runtime on              # 활성화
> /codex-runtime off             # 비활성화 (기본 Hermes 런타임으로 복귀)
> /codex-runtime                 # 현재 상태 확인
```

활성화하면 Hermes가 자동으로 수행하는 작업:

- Codex CLI 설치 여부 확인
- `config.yaml` 에 `model.openai_runtime: codex_app_server` 저장
- Hermes의 MCP 서버를 `~/.codex/config.toml` 에 등록
- 설치된 Codex 플러그인 자동 마이그레이션
- 샌드박스 기본 권한을 `:workspace` 로 설정

변경은 **다음 세션부터** 적용됩니다. 현재 세션은 프롬프트 캐시를 보존하기 위해 기존 런타임을 유지합니다.

사용 가능한 도구의 변화

Codex 런타임에서 모델이 사용할 수 있는 도구는 세 곳에서 옵니다.

1. Codex 내장 도구 — 항상 사용 가능

도구	설명
`shell`	샌드박스 안에서 셸 명령 실행 (파일 읽기/쓰기/검색/빌드 등)
`apply_patch`	구조화된 멀티 파일 diff 적용
`update_plan`	Codex 내부 할 일 관리 (Hermes의 `todo`에 대응)
`view_image`	로컬 이미지를 대화에 로드
`web_search`	Codex 자체 웹 검색

2. Codex 네이티브 플러그인 — 설치한 것만 사용 가능

Linear, GitHub, Gmail, Google Calendar, Canva 등. Hermes가 활성화 시 자동으로 마이그레이션합니다.

3. Hermes 도구 콜백 — MCP를 통해 사용 가능

Codex에 없는 Hermes 고유 도구를 MCP 서버로 노출합니다.

- `web_search` / `web_extract` (Firecrawl 기반)
- `browser_*` (브라우저 자동화 10종)
- `vision_analyze`, `image_generate`
- `skill_view`, `skills_list`
- `text_to_speech`

사용 불가 도구 — 기본 런타임으로 돌아가야 합니다

도구	사유
`delegate_task`	에이전트 루프 컨텍스트 필요 (서브에이전트 생성)
`memory`	에이전트 루프 컨텍스트 필요 (영속 메모리)
`session_search`	에이전트 루프 컨텍스트 필요 (세션 간 검색)
`todo`	에이전트 루프 컨텍스트 필요 (Codex의 `update_plan` 으로 대체)

이 네 도구는 실행 중인 AIAgent의 상태에 접근해야 하므로, 상태 없는(stateless) MCP 콜백으로는 처리할 수 없습니다.

기본 런타임과 비교

항목	기본 런타임	Codex 런타임
ChatGPT 구독 인증	불가	가능
Codex 플러그인 (Linear, GitHub 등)	불가	가능
Codex 샌드박스	불가	가능
`delegate_task` (서브에이전트)	가능	불가
`memory` , `session_search`	가능	불가
메모리·스킬 자동 리뷰	가능	가능
게이트웨이 (텔레그램 등)	가능	가능
`/goal` (Ralph loop)	가능	가능
kanban 다중 에이전트	가능	가능
OpenAI 외 프로바이더	가능	해당 없음

메모리·스킬 자동 리뷰는 Codex 런타임에서도 작동합니다. Codex의 이벤트(`commandExecution` , `fileChange` 등)를 Hermes 메시지 형태로 변환(프로젝션)하여, 리뷰 에이전트가 평소와 같은 대화 기록을 보게 됩니다.

샌드박스 권한

Codex는 세 가지 권한 프로파일을 제공합니다.

프로파일	설명
`:read-only`	쓰기 불가, 모든 명령에 승인 필요
`:workspace`	현재 작업 디렉터리 내 쓰기는 승인 없이 허용 **(기본값)**
`:danger-no-sandbox`	샌드박스 없음 (권장하지 않음)

Hermes가 런타임을 활성화할 때 `:workspace`를 기본으로 설정하므로, 작업 디렉터리 안에서의 파일 쓰기는 매번 승인을 요청하지 않습니다.

신뢰성 — 세션 재사용·자동 회수·OAuth 갱신

장시간 에이전트 실행에서 자주 발목을 잡던 부분이 다음과 같이 보강돼 있습니다.

- **세션 재사용**: 같은 Codex 세션을 여러 호출에 걸쳐 유지하므로 새 세션을 띄우는 오버헤드가 줄어듭니다.
- **위치 잡힌 세션 자동 회수**: 응답을 못 받거나 비정상 상태로 빠진 세션을 자동으로 폐기하고 새 세션을 띄웁니다. 무인 운영(`/goal`, 크론)에서 한 세션 고장이 다음 턴까지 번지지 않습니다.
- **OAuth 갱신 분류**: 토큰 만료·재인증 필요·진짜 인증 실패를 구분해 처리합니다. 일시 만료에는 자동 갱신을 시도하고, 진짜 실패만 사용자에게 알립니다.

언제 쓰고, 언제 끄나

Codex 런타임이 유리한 경우:

- ChatGPT 구독이 있고 API 키 비용을 줄이고 싶을 때
- Linear·GitHub·Gmail 같은 Codex 플러그인이 필요할 때
- Codex의 샌드박스 격리가 필요한 코딩 작업

기본 런타임으로 돌아가야 하는 경우:

- 서브에이전트(`delegate_task`)가 필요한 복잡한 작업
- 영속 메모리를 직접 조작해야 할 때
- OpenAI 외의 모델(Claude, Gemini, Ollama 등)을 사용할 때

두 런타임을 오가는 것은 `/codex-runtime on`과 `/codex-runtime off`로 간단하며, 다음 세션부터 적용됩니다.